

Appendix A

C++ Program Code: CMSA Applied to the MDS Problem

The following program code in C++ is the one of CMSA_INT applied to the minimum dominating set (MDS) problem as described in Chap. 1. This program code was used for the experimentation. We provide this code as an example for the simplicity of CMSA.

CMSA for the MDS Problem

```
/* **** */
      cmsa.cpp - description
      -----
begin      : Wed Nov 30 2022
copyright  : (C) 2022 by Christian Blum
email      : christian.blum@iia.csic.es
/* **** */

/* **** */
*   This program is free software; you can redistribute it *
*   and/or modify it under the terms of the GNU General   *
*   Public License as published by the Free Software      *
*   Foundation; either version 2 of the License, or (at   *
*   your option) any later version.                        *
/* **** */

using namespace std;

#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <fstream>
#include <sstream>
#include <cmath>
#include <vector>
```

```

#include <list>
#include <set>
#include <map>
#include <iomanip>
#include <algorithm>
#include <sstream>
#include <limits>
#include <random>
#include <chrono>
#include <ilcplex/ilocplex.h>

ILOSTLBEGIN

struct Option {
    int vertex;
    double value;
};

struct Solution {
    set<int> vertices;
    int score;
};

// CMSA PARAMETERS
double computation_time_limit = 1000.0;
double cplex_time_limit = 10.0;
double determinism_rate = 0.8;
int n_of_sols = 10;
int age_limit = 10;
int candidate_list_size = 10;
bool warm_start = false;
bool heuristic_emphasis = false;
bool cplex_abort = false;

// INSTANCE DATA
int n_of_vertices;
vector< set<int> > neigh;
string input_file;

/* function for making CPLEX abort a run when improving
   over the currently best solution */
ILOSOLVECALLBACK2(abortCallback, IloCplex::Aborter&, abo,
                  int&, curbest) {

    if (hasIncumbent()) {
        IloNum nv = getIncumbentObjValue();
        if (curbest > int(nv)) abo.abort();
    }
}

/* function for sorting a vector of options */
bool option_compare(const Option& o1, const Option& o2) {

    return (o1.value > o2.value);
}

```

```

}

/* function for producing a random integer between 0
and max - 1 */
int produce_random_integer(int max, double& rnum) {

    int num = int(double(max)*rnum);
    if (num == max) num = num - 1;
    return num;
}

/* function that returns a random element from a set of int */
int get_random_element(const set<int>& s, double& rnum) {

    double r = produce_random_integer(int(s.size()), rnum);
    set<int>::iterator it = s.begin();
    advance(it, r);
    return *it;
}

/* function for reading command line parameter values */
void read_parameters(int argc, char **argv) {

    int iarg=1;
    while (iarg < argc) {
        if (strcmp(argv[iarg], "-i")==0) input_file = argv[++iarg]
        ];
        else if (strcmp(argv[iarg], "-t")==0)
            computation_time_limit = atof(argv[++iarg]);
        else if (strcmp(argv[iarg], "-cpl_t")==0)
            cplex_time_limit = atof(argv[++iarg]);
        else if (strcmp(argv[iarg], "-drate")==0)
            determinism_rate = atof(argv[++iarg]);
        else if (strcmp(argv[iarg], "-nsols")==0)
            n_of_sols = atoi(argv[++iarg]);
        else if (strcmp(argv[iarg], "-max_age")==0)
            age_limit = atoi(argv[++iarg]);
        else if (strcmp(argv[iarg], "-lsize")==0)
            candidate_list_size = atoi(argv[++iarg]);
        else if (strcmp(argv[iarg], "-warm_start")==0) {
            int val = atoi(argv[++iarg]);
            if (val == 1) warm_start = true;
        }
        else if (strcmp(argv[iarg], "-h_emph")==0) {
            int val = atoi(argv[++iarg]);
            if (val == 1) heuristic_emphasis = true;
        }
        else if (strcmp(argv[iarg], "-cpl_abort")==0) {
            int val = atoi(argv[++iarg]);
            if (val == 1) cplex_abort = true;
        }
        iarg++;
    }
}

```

```

/* function for solving a sub-instance by calling CPLEX */
void run_cplex(Solution& cpl_sol, Solution& best_sol,
               vector<int>& age, double& r_limit) {

    IloEnv env;
    env.setOut(env.getNullStream());
    env.setWarning(env.getNullStream());
    cpl_sol.score = std::numeric_limits<int>::max();

    try {

        IloModel model(env);

        /* for each vertex of the input graph we introduce a
           binary variable */
        IloNumVarArray x(env, n_of_vertices, 0, 1, ILOINT);

        // preparing warm-start
        IloNumVarArray mipVar(env);
        IloNumArray mipVal(env);
        if (warm_start) {
            for (int i = 0; i < n_of_vertices; ++i) {
                mipVar.add(x[i]);
                if (int((best_sol.vertices).count(i)) > 0) mipVal
                    .add(1);
                else mipVal.add(0);
            }
        }
        // end preparing warm-start

        // generating the objective function
        IloExpr obj(env);
        for (int i = 0; i < n_of_vertices; ++i) obj += x[i];
        model.add(IloMinimize(env, obj));
        obj.end();

        // generating the constraints
        for (int i = 0; i < n_of_vertices; ++i) {
            IloExpr expr(env);
            expr += x[i];
            for (set<int>::iterator sit = neigh[i].begin(); sit
                != neigh[i].end(); ++sit) expr += x[*sit];
            model.add(expr >= 1);
            expr.end();

            /* the values of those variables whose vertices are
               not in the sub-instance are fixed to zero */
            if (age[i] == -1) {
                IloExpr expr1(env);
                expr1 += x[i];
                model.add(expr1 == 0);
                expr1.end();
            }
        }
    }
}

```

```

IloCplex cpl(model);

/* the aborter stops CPLEX once a better solution than
"best_sol" is found */
if (cplex_abort) {
    IloCplex::Aborter abo(env);
    cpl.use(abo);
    cpl.use(abortCallback(env, abo, best_sol.score));
}
if (warm_start) cpl.addMIPStart(mipVar, mipVal);
cpl.setParam(IloCplex::TiLim, r_limit);
cpl.setParam(IloCplex::EpGap, 0.0);
cpl.setParam(IloCplex::EpAGap, 0.0);
cpl.setParam(IloCplex::Threads, 1);
if (heuristic_emphasis) cpl.setParam(IloCplex::Param::
    Emphasis::MIP, 5);
cpl.setWarning(env.getNullStream());

// calling CPLEX to solve the model
cpl.solve();

/* the following is done if CPLEX found at least one
feasible solution */
if (cpl.getStatus() == IloAlgorithm::Optimal
    or cpl.getStatus() == IloAlgorithm::Feasible) {
    cpl_sol.score = 0;
    IloNumArray x_val(env);
    cpl.getValues(x_val, x);
    for (int i = 0; i < n_of_vertices; ++i) {
        // ADAPT-STEP of CMSA
        if (age[i] >= 0) {
            /* increment the age of all vertices in the
            sub-instance */
            age[i] += 1;
            if (double(x_val[i]) > 0.8) {
                cpl_sol.score += 1;
                /* set the age of all vertices from the
                best CPLEX solution to zero */
                age[i] = 0;
                (cpl_sol.vertices).insert(i);
            }
            /* remove all vertices whose age has reached
            the age limit from the sub-instance */
            if (age[i] >= age_limit) {
                age[i] = -1;
            }
        }
    }
}

}

catch (IloException& e) {
    cerr << "Concert exception caught: " << e << endl;
}

env.end();

```

```

}

/* function that probabilistically generates a solution with a
   greedy bias */
void generate_solution(Solution& greedy_sol, vector<int>& age,
    default_random_engine& generator,
    uniform_real_distribution<double>& standard_distribution) {

    greedy_sol.score = 0;
    vector<bool> already_covered(n_of_vertices, false);
    int num_nodes_uncovered = n_of_vertices;
    vector< set<int> > uncovered_neighbors = neigh;

    set<int> candidates;
    for (int i = 0; i < n_of_vertices; ++i) candidates.insert(i);

    while (num_nodes_uncovered > 0) {
        vector<Option> choice;
        double max_val = -1.0;
        set<int> max_vertices;

        /* generate all options for the extension of the current
           partial solution */
        for (set<int>::iterator cit = candidates.begin();
            cit != candidates.end(); ++cit) {
            Option opt;
            opt.vertex = *cit;
            opt.value = double(uncovered_neighbors[*cit].size());
            if (opt.value >= max_val) {
                if (opt.value > max_val) {
                    max_val = opt.value;
                    max_vertices.clear();
                }
                max_vertices.insert(*cit);
            }
            choice.push_back(opt);
        }
        sort(choice.begin(), choice.end(), option_compare);

        int chosen_vertex;
        double dec = standard_distribution(generator);
        if (dec > determinism_rate) {
            int max = candidate_list_size;
            if (int(choice.size()) < candidate_list_size)
                max = int(choice.size());
            double rnum = standard_distribution(generator);
            int pos = produce_random_integer(max, rnum);
            chosen_vertex = choice[pos].vertex;
        }
        else {
            double rnum = standard_distribution(generator);
            chosen_vertex = get_random_element(max_vertices, rnum);
        }
    }
}

```

```

        (greedy_sol.vertices).insert(chosen_vertex);

/* MERGE-STEP of CMSA: if a vertex chosen for the current
   solution does not yet form part of the sub-instance,
   add it to the sub-instance by initializing its age
   value to zero */
if (age[chosen_vertex] == -1) {
    age[chosen_vertex] = 0;
}

if (not allready_covered[chosen_vertex]) {
    allready_covered[chosen_vertex] = true;
    --num_nodes_uncovered;
}
greedy_sol.score += 1;

num_nodes_uncovered -= int(uncovered_neighbors[
    chosen_vertex].size());
set<int> to_cover = uncovered_neighbors[chosen_vertex];
for (set<int>::iterator sit = to_cover.begin();
     sit != to_cover.end(); ++sit) {
    allready_covered[*sit] = true;
    for (set<int>::iterator ssit = neigh[*sit].begin();
         ssit != neigh[*sit].end(); ssit++)
        uncovered_neighbors[*ssit].erase(*sit);
}
uncovered_neighbors[chosen_vertex].clear();

for (set<int>::iterator sit = neigh[chosen_vertex].begin()
     (); sit != neigh[chosen_vertex].end(); sit++)
    uncovered_neighbors[*sit].erase(chosen_vertex);
candidates.erase(chosen_vertex);
set<int> to_delete;

for (set<int>::iterator cit = candidates.begin(); cit !=
     candidates.end(); ++cit) {
    if (int(uncovered_neighbors[*cit].size()) == 0 and
        allready_covered[*cit]) to_delete.insert(*cit);
}
for (set<int>::iterator sit = to_delete.begin(); sit !=
     to_delete.end(); ++sit) candidates.erase(*sit);
}
}

/*****
Main function
*****/

int main( int argc, char **argv ) {

    if (argc < 3) {
        cout << "Usage: ./cmsa -i <input_file> ..." << endl;
        exit(1);
    }
}

```

```

}
else read_parameters(argc,argv);

std::cout << std::setprecision(2) << std::fixed;

// initializes the random number generator
unsigned seed = std::chrono::system_clock::now().
    time_since_epoch().count();
std::default_random_engine generator(seed);
std::uniform_real_distribution<double>
    standard_distribution(0.0,1.0);

ifstream indata;
indata.open(input_file.c_str());
if(!indata) {
    cout << "Error: file could not be opened" << endl;
}

// reading the problem instance file
indata >> n_of_vertices;
neigh = vector< set<int> >(n_of_vertices);
int v1, v2;
while (indata >> v1 >> v2) {
    neigh[v1].insert(v2);
    neigh[v2].insert(v1);
}
indata.close();

/* "age" is an integer vector that contains the age of all
   vertices. An age of -1 means that the vertex does not form
   part of the sub-instance, while an age of >= 0 means that
   the vertex forms part of the sub-instance. */
vector<int> age(n_of_vertices, -1);

Solution best_sol;
best_sol.score = std::numeric_limits<int>::max();

// the computation time starts now
clock_t start = clock();

// variable ctime stores the current time that has passed
double ctime = 0.0;

bool stop = false;

// main loop of the CMSA algorithm
while (not stop and (ctime < computation_time_limit)) {

    // CONSTRUCT-STEP of CMSA: generate "n_of_sols" solutions
    for (int na = 0; na < n_of_sols; ++na) {
        Solution greedy_sol;
        generate_solution(greedy_sol, age, generator,
            standard_distribution);
        if (greedy_sol.score < best_sol.score) {

```

```

        best_sol = greedy_sol;
        clock_t current = clock();
        ctime = double(current - start) / CLOCKS_PER_SEC;
        cout << "best " << best_sol.score << "\ttime " <<
            ctime << "\tgreedy" << endl;
    }
}

/* calculate the time "r_limit" given to CPLEX for the
next application to the current sub-instance */
clock_t current = clock();
ctime = double(current - start) / CLOCKS_PER_SEC;
double r_limit = computation_time_limit - ctime;
if (r_limit > cplex_time_limit) r_limit =
    cplex_time_limit;

/* if the remaining computation time is less than 0.1
seconds, it does not make sense to call CPLEX.
Variable 'stop' is set to true and the algorithm stops */
if (r_limit < 0.1) stop = true;
if (not stop) {
    Solution cpl_sol;
    /* SOLVE-STEP of CMSA: apply CPLEX to the current
sub-instance */
    run_cplex(cpl_sol, best_sol, age, r_limit);
    if (cpl_sol.score < best_sol.score) {
        best_sol = cpl_sol;
        current = clock();
        ctime = double(current - start) / CLOCKS_PER_SEC;
        cout << "best " << best_sol.score << "\ttime " <<
            ctime << "\tcpex" << endl;
    }
    current = clock();
    ctime = double(current - start) / CLOCKS_PER_SEC;
}
}
}

```
